Roadmap: step by step

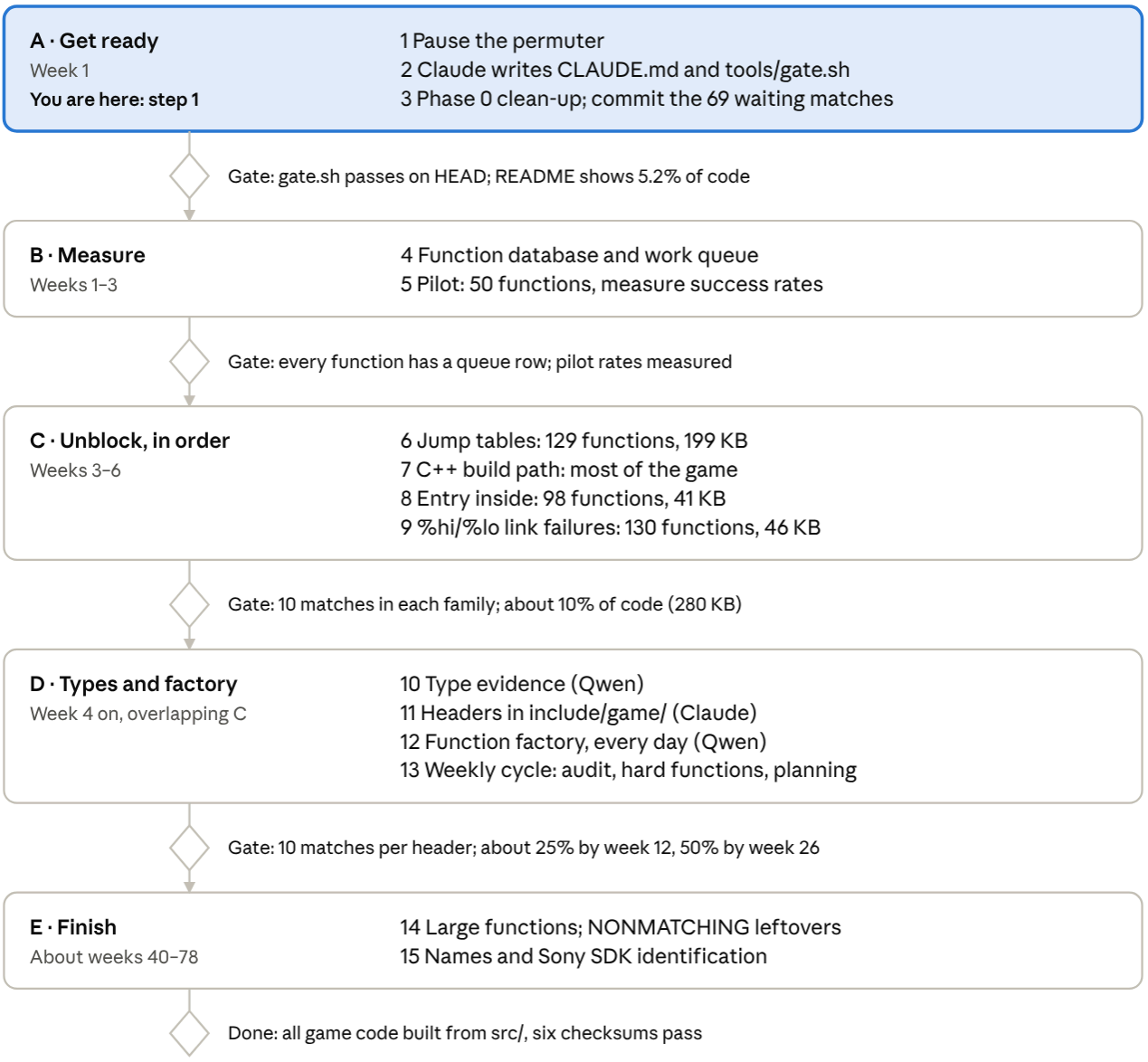
Oct 13, 2026 · @user

Fifteen steps in five stages take the decompilation from 5.2% of game code (146 KB) to all of it in about 12–18 months. Do them in order; each stage ends at a gate you can check. You are at step 1.

Each step says who runs it and which ready-to-paste prompt to use (C1–C10 for Claude, Q1–Q6 for Qwen, in the Runbook tab). Set the Status column as you go.

At a glance

Fifteen steps, five gates: from 5.2% to all of the game code



roadmap · 5 stages, 15 steps, 5 gates

Read top to bottom: each diamond is a gate that must pass before the next stage starts. Stage D starts in week 4, while C is still running.

Stage A: get ready (week 1)

Set the rules and the safety net before more code goes in. **Gate:** `tools/gate.sh` passes on HEAD, and the README shows 29.8% of functions and 5.2% of code.

#	Step	Who and prompt	Done when	Status
1	Pause the permuter (PID 8231, running since this morning). Leave the 69 untracked match files in `src/` where they are.	Qwen: **Q6** (or `kill $(cat build/permute_batch.pid)`)	No `permute_batch` process is running	Not started
2	Write the house rules and the safety net: `CLAUDE.md`, `tools/gate.sh`, and a hook that blocks commit/push unless the gate passes.	Claude: **C1**	The gate passes on HEAD and refuses a planted bad file	Not started
3	Phase 0 clean-up: `readme_progress.py` counts bytes and skips `asmfunc/`; fix `tools/watch.py`; tag the 20 contrived permuter files `// permuter-artifact`; send the 69 untracked matches through the gate and commit the ones that pass.	Qwen: **Q1**, working the task file C1 wrote	The README shows 29.8% / 5.2%; `git status` is clean	Not started

Stage B: measure (weeks 1–3)

List every remaining function, then time a small sample so the rest of the plan runs on real rates, not guesses. **Gate:** every remaining function has a queue row and a blocker class, and the pilot rates are written down.

#	Step	Who and prompt	Done when	Status
4	Build `tools/funcdb.py` and the work queue `build/queue.tsv`: size, callers, strings, translation-unit guess and blocker class for each function.	Claude: **C2** writes the task; Qwen: **Q1** builds it	About 7,900 rows, one per remaining function	Not started

#	Step	Who and prompt	Done when	Status
5	Pilot: 50 functions across all five size buckets. Qwen takes 40, Claude 10; record attempts, time and success.	Claude: **C3** picks, **C4** does its 10, **C5** reviews; Qwen: **Q2** `tasks/pilot.md`	Success rate per size bucket is measured and the timeline re-estimated	Not started

Decision at step 5: if Qwen matches fewer than 20% of the 65–256 B pilot functions, move Claude time to types (step 10) before scaling the factory.

Stage C: unblock, in this order (weeks 3–6)

Fix the build so whole families of functions can match at all, biggest payoff first. For each one, Claude designs the fix and proves it on 3 functions (**C6**), then Qwen applies it across the family (**Q1**). **Gate:** at least 10 matched functions in each family.

#	Family	Left (13 Oct)	The fix	Done when	Status
6	**Jump tables**	129 functions, 199 KB	Place each function's `.rodata` next to it (`gen_configs.py` , linker script); let `autodecomp.py` keep and compare `.rodata` instead of rejecting it	10 jump-table functions match	Not started
7	**C++ build path**	most of the game	`.cpp` files: classes, bit fields, base-class casts, name mangling or `extern "C"`	10 C++ matches, including a virtual call and a base-class cast; the 20 permuter-artifact files rewritten as clean C++	Not started
8	**Entry inside a function**	98 functions, 41 KB	Split each range at its inner entry point in the config	10 split functions match	Not started
9	**`%hi/%lo` link failures**	130 functions, 46 KB	Real C with `&sym` and correct externs	10 functions match	Not started

Jump tables come first because no amount of matching effort can succeed on them until the build accepts `.rodata` , and they hold more code (199 KB) than everything matched so far (146 KB). Float constants are not a step of their own: only 1 function failed on them, and step 6 lets literals be written naturally.

Stage D: types and the function factory (week 4 onward)

This is the long middle, where most of the code gets matched. Steps 10–13 run together every week until only large functions are left. **Gate per header:** at least 10 functions that use it match. **Milestones:** about 25% of code (700 KB) by week 12, and about 50% by week 26.

#	Step	Who and prompt	Done when	Status
10	Collect type evidence: vtables, constructors, field accesses per object, `__FILE__` and assert strings.	Qwen: **Q4**, when a types task asks for it	Evidence files exist for the most-called classes	Not started
11	Turn the evidence into headers under `include/game/`: class hierarchy, struct layouts, bit fields, names.	Claude: **C8**, Tuesday and Wednesday	Each header has 10 matching users	Not started
12	Run the function factory: next queue row, draft in C++ with the headers, compile and diff, stop after 30 attempts or 45 minutes, mark failures `needs-claude`.	Qwen: **Q3**, every day the laptop is on	Ongoing; matches commit only through the gate	Not started
13	Keep the weekly cycle going: Monday audit, Thursday hard functions, Friday task files. You restart Qwen daily and approve one push.	Claude: **C7**, **C9**, **C10**; Qwen: **Q1** and **Q5**; you	Each Monday audit passes	Not started

If the Pro weekly limit runs out early, drop Thursday's hard functions first. Types and the audit come first.

Stage E: finish (about weeks 40–78)

Close out what the factory couldn't do, then make the code readable. **Gate:** every game function is built from `src/` and the six checksums pass.

#	Step	Who and prompt	Done when	Status
14	Large and stubborn functions (over 1 KB). Claude solves them; Qwen gets the asm in labelled chunks. What still won't match is kept as `NONMATCHING` C beside its asm.	Claude: **C9** (Opus for the biggest); Qwen: **Q2**	No game function is left in asm only	Not started
15	Readability and SDK: rename functions and fields from strings and context; identify the ~990 Sony SDK functions against known library versions instead of decompiling them.	Qwen renames; Claude reviews in **C7**	The README shows 100% of game code	Not started

DQ8 Decompilation Plan: Claude Pro + Local Qwen

Oct 13, 2026 · @user

Goal and starting point

The goal is to turn the remaining 2,682 KB of game code into C++ that rebuilds byte for byte. Qwen does the volume. Claude Pro does the thinking that unlocks it.

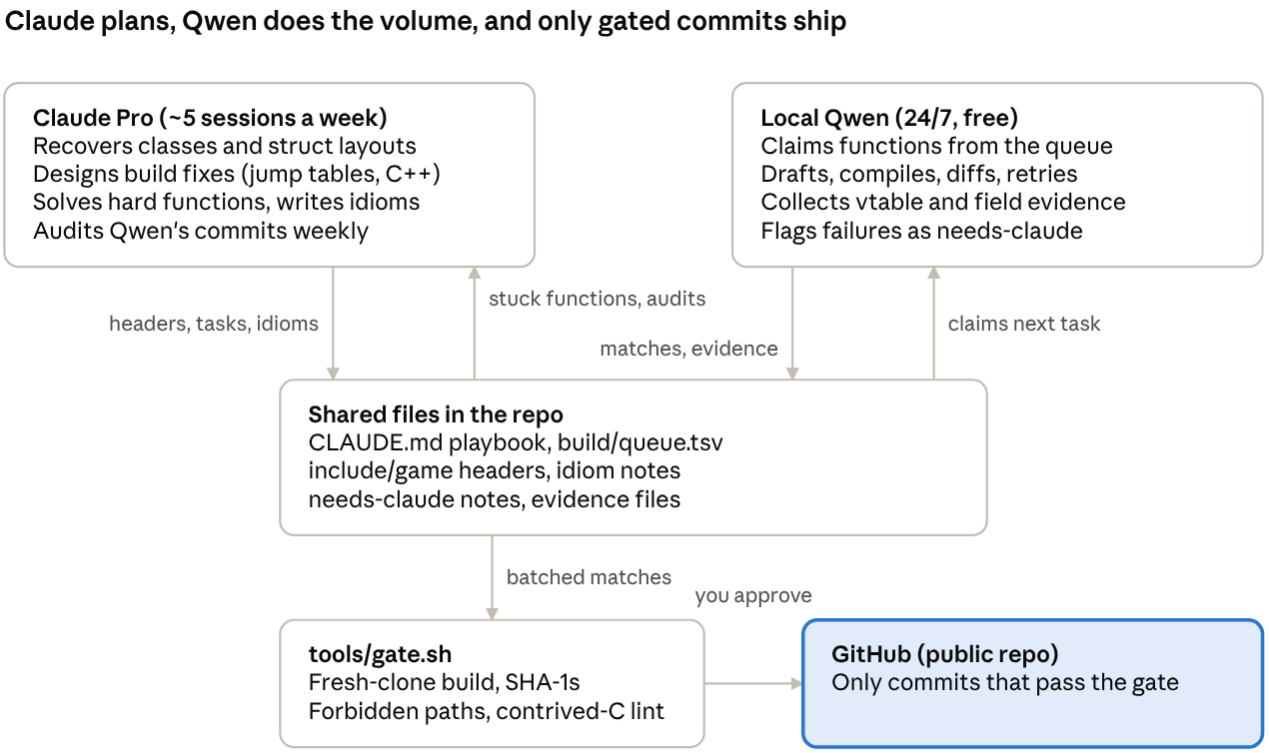
	Functions	Code
Matched (in git)	3,451 / 11,575 (29.8%)	146 / 2,828 KB (5.2%)
Left: ⩽ 64 B	1,703	80 KB
Left: 65–256 B	3,641	544 KB
Left: 257 B–1 KB	2,321	1,094 KB
Left: 1–4 KB	424	724 KB
Left: > 4 KB	35	240 KB

"Done" means every game function is built from `src/` and the six checksums pass. A few functions may never match exactly; those can be kept as `NONMATCHING` C next to their asm. Progress is tracked in **code bytes**, because function counts flatter tiny functions. The Sony SDK (GCC-built, ~990 functions) is out of scope: it is identified, not decompiled.

The automated passes (m2c drafts, thunkify, blind permuter) have mined the easy pool. The current permuter run matches about 4 in 100. Everything left needs types, C++ idioms or build fixes first.

Division of labour

Claude gives Qwen its instructions through files in the repo, which Qwen reads at the start of every session. A script decides what ships.



work split · Claude Pro, local Qwen, the gate

You restart Qwen after the laptop sleeps and approve one batch of pushes a day.

Day-by-day schedule and ready-to-paste prompts for both sessions:
Runbook: schedule and prompts

Budgets

Claude Pro is the scarce resource, at roughly 40–80 Sonnet-hours a week (third-party estimate; check `/status`). Spend it only on work that unlocks many functions at once. Qwen is free and runs whenever the laptop is on.

Claude Pro, per week:

Work	Share	Why Claude
Type and class recovery (headers)	40%	Interpreting vtables and field patterns needs judgment; one header unlocks dozens of functions
Hard functions (> 1 KB, stuck) + idiom notes	25%	Large asm overflows Qwen's context; each solved idiom becomes a rule Qwen reuses

Work	Share	Why Claude
Build-class fixes (front-loaded in weeks 1–6)	15%	Jump tables, C++ path, entry points, %hi/%lo: design once, Qwen applies everywhere
Weekly audit of Qwen's commits	10%	Same check as the 2026-10-13 review
Writing next week's tasks and playbook updates	10%	Keeps Qwen productive without supervision

Rules: keep Claude out of mechanical loops (retries, permuter runs, bulk compiles). Avoid heavy claude.ai chat, since it draws from the same pool. Use the cheaper model for audits.

Qwen, from your llama.cpp logs: prompt processing ~600 tokens/s, generation ~15 tokens/s, contexts up to ~28K tokens. One match attempt (read a ~5K-token diff, write ~400 tokens of C, compile) takes about 40 s. That's ~90 attempts an hour, or ~1,400 a day at 16 hours of uptime. If a typed medium-size function needs ~20 attempts and half of them match, Qwen produces roughly 25–50 matches a day. The pilot (Phase 2) will replace this guess with a measured rate.

Phases

Six phases, overlapping from week 4 on. Each one ends at a gate that can be checked. The Roadmap tab turns them into 15 numbered steps.

Phase 0 — Hygiene (week 1)

1. Claude writes `CLAUDE.md`: the rules Qwen reads at the start of every session (verification, no contrived C, commit rules, progress in bytes).
2. Claude writes `tools/gate.sh`. It clones HEAD into a temp directory, runs `make split && make`, checks the six SHA-1s, refuses forbidden paths, and lints for contrived patterns.
3. Qwen fixes `readme_progress.py` (count bytes, exclude `asmfunc/`) and `tools/watch.py`, and tags the 20 contrived permuter files `// permuter-artifact`.

Gate: `gate.sh` passes on HEAD, and the README shows 29.8% of functions and 5.2% of code.

Phase 1 — Function database and work queue (weeks 1–2)

1. Qwen builds `tools/funcdb.py`. For every function it records binary, address, size, callers, callees, referenced strings, translation-unit guess and blocker class: jump table, `%hi/%lo`, entry inside, or none.

2. Qwen generates the queue `build/queue.tsv` with the states `todo`, `claimed`, `matched`, `stuck` and `needs-claude`, ordered by translation unit, then unblocked first, then smallest first.

Gate: every remaining function has a row and a blocker class.

Phase 2 — Pilot (weeks 2–3)

1. Pick 50 functions across all five size buckets.
2. Qwen attempts 40 of them and Claude 10, both following the playbook.
3. Record attempts, wall time, success rate, and Claude usage per function.

Gate: the timeline is re-estimated from measured rates. If Qwen's success rate on 65–256 B functions is below 20%, more Claude time goes to types before scaling up.

Phase 3 — Unblock whole classes of functions (weeks 3–6)

Claude designs each fix in one session. Qwen implements it and regression-tests it on the whole tree.

Order	Blocker	Left (13 Oct)	Fix
1	Jump tables	129 functions, 199 KB	Place each function's `.rodata` with it in `gen_configs.py` and the linker script; let `autodecomp.py` keep and compare `.rodata`
2	C++ features	most of the game	A `.cpp` build path: classes, bit fields, base-class casts, name mangling or `extern "C"`
3	Entry point inside a function	98 functions, 41 KB	Split those ranges by hand
4	Absolute `%hi/%lo` data (link failures)	130 functions, 46 KB	Real C with `&sym` and correct externs

Jump tables go first: every one of them is blocked by the build, and they hold more code than everything matched so far. Float constants were dropped as a blocker: only 1 function failed on them, and the jump-table fix lets literals be written naturally.

Gate: every class has at least 10 matched examples.

Phase 4 — Types and classes (weeks 4–16, then ongoing)

1. Qwen collects the raw evidence:

- vtables (runs of words in `.data` pointing at function starts)
- constructors (code storing a vtable address at offset 0)
- field accesses grouped per object (offset, width, int or float, read or write)
- assert and `__FILE__` strings mapped to translation units

2. Claude turns the evidence into headers under `include/game/`: class hierarchy, struct layouts, bit fields, and names taken from strings. The most-called classes come first.
3. Qwen re-drafts every function that uses a new header.

Gate per header: at least 10 functions that use it match.

Phase 5 — Function factory (week 5 to the end)

Qwen runs this loop around the clock. Claude handles what falls out of it.

1. Claim the next `todo` function from the queue, grouped by translation unit.
2. Draft it as C++ with the real headers.
3. Compile, diff against the original asm (`tools/try_match.py --show`), and fix. Stop after 30 attempts or 45 minutes.
4. Use the permuter only as a finisher, when the diff is already small.
5. On a match, the lint must pass. Batch matches and commit only through `gate.sh`.
6. If it fails, mark it `needs-claude` and attach the best diff and notes.
7. Each week Claude solves a sample of `needs-claude` functions and writes the idiom it found into the playbook. Qwen then retries that whole class.

Phase 6 — Readability and SDK (last)

Rename functions and fields from strings and context. Identify Sony SDK functions against known library versions instead of decompiling them.

Weekly cadence

Claude works in about five focused sessions a week. Qwen works whenever the laptop is on. You spend about 30 minutes a day.

When	Who	What
Monday	Claude	Audit last week's commits: fresh-clone build, a random sample of 20 new files, lint results. Reprioritize the queue.

When	Who	What
Tuesday, Wednesday	Claude	Type and class recovery: turn Qwen's evidence files into headers, and queue re-drafts.
Thursday	Claude	Hard functions from `needs-claude`. Each new idiom goes into the playbook.
Friday	Claude	Write next week's task files and update `CLAUDE.md`.
Every day	Qwen	Function factory, evidence collection, regression runs of `gate.sh`.
Every day	You	Restart Qwen after the laptop sleeps, check `tools/watch.py`, approve pushes.

If the Pro weekly limit runs out early, the Thursday hard-function session is the one to drop. Types and the audit come first.

Timeline and milestones

The whole game should take about 12–18 months, against 1–2 years or more on Claude Pro alone. These are pre-pilot guesses; week 3 replaces them with measured rates.

Week	Milestone	Code matched
1	Hygiene done, honest README, gate in place	5.2% (146 KB)
3	Pilot measured, timeline re-estimated	~6%
6	Jump tables and the C++ path work	~10% (~280 KB)
12	Core classes typed; most functions up to 256 B matched	~25% (~700 KB)
26	Half of the 257 B–1 KB functions matched	~50%
40–52	Only large and stubborn functions left	~85–95%
52–78	Everything built from source; leftovers kept as `NONMATCHING`	~100%

The biggest swing factor is Qwen's success rate on typed medium functions. At 25 matches a day the middle phase takes about 8 months. At 50 a day it takes about 4.

Quality gates and risks

Nothing reaches GitHub unless `tools/gate.sh` passes. The gate is a script, so it doesn't depend on Qwen's judgment.

The gate checks:

- A fresh clone of the commit builds, and all six SHA-1s match the ISO.
- No game data or binaries are staged.
- Lint: no `volatile` outside hardware registers, no `do { } while (0)`, no comma-expression tricks, no unused `pad` locals, no pasted typedef blocks. Every file includes `common.h`.
- Progress is reported in bytes, from committed files only.
- Each commit contains only what its message describes: `src/` and tools in separate commits.

Risk	Likelihood	Mitigation
Qwen's quality drifts over long sessions	High	Gate script, weekly Claude audit, rules restated in `CLAUDE.md` at every session start
Functions over ~1 KB overflow Qwen's ~28K context	High	Route them to Claude, or give Qwen the asm in labelled chunks
Pro limits tighter than estimated	Medium	Claude does types and audits only; Qwen takes all matching
Laptop downtime	Certain	The queue is resumable; `claimed` rows older than a day return to `todo`
A compiler quirk blocks a whole class	Medium	Allow `NONMATCHING` C beside the asm; revisit later
C++ name mangling breaks links	Medium	Prove the `.cpp` path in Phase 3 before scaling
Your time becomes the bottleneck	Medium	Batch approvals: one review and push a day

Runbook: schedule and prompts

How the hand-off works

Claude does instruct Qwen, by writing the instructions into files that Qwen reads by itself. You only start each session with a short prompt that points at those files.

File	Written by	Read by	Holds
`CLAUDE.md`	Claude	Qwen, automatically at every session start	The standing rules
`.claude/skills/match-function/`	Claude	Qwen, when matching	The step-by-step matching loop and the CodeWarrior idiom notes
`tasks/NEXT.md` + `tasks/*.md`	Claude	Qwen	This week's tasks, each with steps and a pass/fail check
`build/queue.tsv`	Qwen (Claude reprioritizes)	Both	One row per function: `todo`, `claimed`, `matched`, `stuck`, `needs-claude`
`notes/needs-claude/*.md`	Qwen	Claude	What Qwen couldn't solve, with its best diff
`notes/weekly/*.md`	Both	You	Short reports: what was done and what's next
`.claude/settings.json` hooks	Claude	the harness	Blocks `git commit` / `git push` unless `tools/gate.sh` passes

The prompts below are short on purpose. The detail lives in the files, so the instructions survive Qwen's context limit and laptop restarts. Prompt IDs: **C** for the Claude Pro session, **Q** for the Qwen session.

One-time setup

You need two terminals, both in `~/Code/DQ8_decompil`.

Terminal	How to start it	Model
A: Claude	`claude` (your Pro login)	Sonnet by default (`/model sonnet`); Opus only when a prompt below says so
B: Qwen	However you start your llama.cpp-backed Claude Code today	qwen3.8-flash-next

Ground rules:

- **One job per Claude session.** Start Claude fresh (`claude`, or `/clear`) for each prompt below. Long sessions burn Pro usage on re-read context.
- **Qwen restarts freely.** Open a new Qwen session whenever its context fills up or the laptop wakes; prompt Q1 rebuilds everything from the files.
- **Only you push.** Both sessions stop at a green `tools/gate.sh` and wait for you.
- **Don't run both on `src/` at once.** When Claude works in `src/`, pause Qwen's factory with Q6.

Daily routine

About 15 minutes a day: start Qwen in the morning, collect its work in the evening, push.

When	Where	Do
Morning, after waking the laptop	Terminal B	Start a new Qwen session and paste **Q1**. It picks up where it stopped.
Morning	Any terminal	Glance at `tools/watch.py --once` (from week 1, once Qwen has fixed it).
Day, per the calendar	Terminal A	Paste that day's **C** prompt into a fresh Claude session. Skip days with none.
Evening	Terminal B	Paste **Q5**. Qwen stops, runs the gate, commits and writes a short report.
Evening	Your own terminal	Read the report. If the gate is green, run `git push`. If it's red, leave it; Monday's audit picks it up.

When	Where	Do
Before closing the laptop	nothing	Nothing needed. Claims older than a day go back to the queue automatically.

Week-by-week calendar

Weeks 1–5 set things up. From week 6 on, every week looks the same. On any day not listed, Qwen just runs Q1 (or Q3 once the factory starts) and Q5 in the evening.

Week	Day	Claude (terminal A)	Qwen (terminal B)
1	Mon	**C1**: Phase 0 setup	Morning: **Q6** to stop the current permuter run first
1	Tue–Wed	—	**Q1**: works the Phase 0 tasks C1 wrote
1	Thu	**C2**: write Phase 1 tasks (function database, queue)	**Q1**
1	Fri–Sun	—	**Q1**: builds the database and queue
2	Mon	**C3**: pick the 50 pilot functions	**Q1**, then **Q2** `tasks/pilot.md` (its 40)
2	Wed, Thu	**C4**: Claude's 10 pilot functions (one session each, 5 functions per session)	**Q2** `tasks/pilot.md`
3	Mon	**C5**: pilot review and re-estimate	**Q4**: collect type evidence
3	Tue	**C6** with `jump tables`	**Q1**: implements the fix
3	Thu	**C6** with `the C++ build path`	**Q1**
4	Mon	**C7**: audit	**Q1**
4	Tue	**C6** with `entry points inside functions`	**Q1**
4	Wed	**C8**: first types session	**Q1**
4	Thu	**C6** with `absolute %hi/%lo data`	**Q1**
4	Fri	**C10**: next week's tasks	**Q1**
5	Mon	**C7**	**Q3**: the function factory starts

Week	Day	Claude (terminal A)	Qwen (terminal B)
5	Tue	**C6** again for any family still under 10 matches; otherwise **C8**: types	**Q3**
5	Wed	**C8**	**Q3**
5	Thu	**C9**: hard functions	**Q3**
5	Fri	**C10**	**Q3**
6+	Mon	**C7**: audit (first Monday of the month: with the re-estimate line)	**Q3**
6+	Tue, Wed	**C8**: types	**Q3**; **Q4** when a C8 task asks for evidence
6+	Thu	**C9**: hard functions	**Q3**
6+	Fri	**C10**: next week's tasks	**Q3**

If the Pro weekly limit runs out, drop C9 first, then the second C8. Never skip C7 and C10: without them Qwen runs unchecked and runs out of tasks.

Claude prompts (terminal A)

Start a fresh Claude session for each prompt. Replace anything in `<angle brackets>`. Every prompt that changes the repo ends by waiting for your OK.

C1 — Phase 0 setup (week 1, Monday)

```
Phase 0 of our decompilation plan: https://claude.ai/code/artifact/9a914f92-5847-4a31-a1fa-91159469eb66 (read the main tab and the Runbook tab). Set up the hand-off between you and the local Qwen session that also runs Claude Code in this repo. Create:
1. tasks/PLAN.md: a one-page summary of the plan (phases, gates, and the file conventions in the Runbook's hand-off table) so neither session needs the doc again.
2. CLAUDE.md with the standing rules for Qwen: a function counts only if autodecomp.verify() says match and tools/gate.sh passes; no contrived C (volatile tricks, do {} while (0), comma expressions, unused pad locals, pasted typedef blocks; always include common.h); src/ and tool changes in separate commits, never git add -A; never push; progress is reported in bytes from committed files; work the first open task in tasks/NEXT.md; when stuck, write notes/needs-claude/<func>.md with the best diff; log to notes/weekly/.
3. tools/gate.sh: clone HEAD into a temp dir, copy iso/ and extract/, link .toolchain and .venv, run make split && make, check the six SHA-1s, refuse forbidden paths (asm/ iso/ extract/ assets/ build/ linker/ asmfunc, any binary), and lint files changed since origin/main for the contrived patterns. Non-zero exit on any failure.
4. A PreToolUse hook in .claude/settings.json that blocks git commit and git push unless tools/gate.sh passed for the current state.
5. .claude/skills/match-function/SKILL.md: the Phase 5 loop (claim from build/queue.tsv, draft, compile, diff with tools/try_match.py --show, at most 30 attempts or 45 minutes, permuter only as a finisher, needs-claude note on failure), plus an idioms section that starts with bit-field stores and base-class pointer casts.
6. tasks/NEXT.md and task files for Qwen: make tools/readme_progress.py count bytes and exclude asmfunc/; fix tools/watch.py; tag the 20 contrived permuter files with // permuter-artifact (list them).
Run tools/gate.sh on the current HEAD and show me the result. Show me the full diff and wait for my OK before committing. Don't touch src/ beyond the tags.
```

C2 — Phase 1 tasks (week 1, Thursday)

```
Read tasks/PLAN.md and the latest notes/weekly/ files. Write the Phase 1
tasks for Qwen in tasks/ and list them in tasks/NEXT.md. Don't implement them
yourself.
(a) tools/funcdb.py: one row per game function with binary, address, size,
callers, callees, referenced strings, translation-unit guess from
assert/__FILE__ strings, blocker class (jump table, %hi/%lo, entry inside,
none) and current status.
(b) build/queue.tsv generated from it, with the states
todo/claimed/matched/stuck/needs-claude, ordered by translation unit, then
unblocked first, then smallest first.
(c) A check that every remaining function has a row and a blocker class,
printing the count per blocker.
Give exact file formats and a pass/fail check for each task, and keep each
task small enough for a 28K-token context. Show me the diff and wait for my
OK before committing.
```

C3 — Pick the pilot (week 2, Monday)

```
Read tasks/PLAN.md and the blocker counts in notes/weekly/. Pick 50 pilot
functions from build/queue.tsv: 10 per size bucket (<=64 B, 65-256 B, 257 B-1
KB, 1-4 KB, >4 KB), unblocked only, spread over several translation units.
Assign 40 to Qwen and 10 to Claude (2 per bucket). Write tasks/pilot.md with
the list, the rules (follow .claude/skills/match-function; record per
function: attempts, minutes, result, best diff score) and the log format
notes/pilot.tsv. Add it to tasks/NEXT.md. Show me the diff and wait for my OK
before committing.
```

C4 — Claude's pilot functions (week 2, Wednesday and Thursday)

```
Pilot session. Read tasks/pilot.md and match the next 5 functions assigned to
Claude that aren't done yet, following .claude/skills/match-function. Log
each one in notes/pilot.tsv (attempts, minutes, result). Add any CodeWarrior
idiom you discover to the skill's idioms section. Run tools/gate.sh, show me
the diff and wait for my OK before committing.
```

C5 — Pilot review (week 3, Monday)

```
Read notes/pilot.tsv. Per size bucket, compute Qwen's and Claude's success rate and minutes per function, and Qwen's matches per day. Re-estimate the remaining timeline from those rates (remaining counts per bucket are in tasks/PLAN.md). Write notes/weekly/<date>-pilot-review.md with the measured rates, the new estimate and any change to the plan (for example more Claude time on types if Qwen's 65-256 B success rate is under 20%), and update tasks/PLAN.md. Give me a five-line summary, then show me the diff and wait for my OK before committing.
```

C6 — Unblock a class of functions (weeks 3–5; pause Qwen with Q6 first)

```
Phase 3, blocker: <jump tables | the C++ build path | entry points inside functions | absolute %hi/%lo data>. Read tasks/PLAN.md and build/queue.tsv. Design the fix that lets this whole class match: what changes in tools/gen_configs.py, the linker scripts, tools/autodecomp.py or the headers. Prove it on 3 functions of the class; they must pass tools/gate.sh. Then write tasks/<blocker>.md for Qwen: apply the fix across the class, regression-test with tools/gate.sh, and record the matches. Add it to tasks/NEXT.md. Show me the diff and wait for my OK before committing.
```

C7 — Weekly audit (every Monday from week 4; Sonnet)

```
Weekly audit. Review everything committed since last Monday, the same way as the 2026-10-13 review: run tools/gate.sh; check that each commit's claimed counts match its files; read a random sample of 20 new src files for contrived or meaningless C; look for patterns in notes/needs-claude/; find stale claims in build/queue.tsv. Don't change src/ or tools/. Write notes/weekly/<date>-audit.md with the findings and the bytes matched this week, add any fix tasks to tasks/NEXT.md, and give me a five-line summary. If this is the first Monday of the month, also recompute the timeline from this month's rate. Show me the diff and wait for my OK before committing.
```

C8 — Types (Tuesday and Wednesday from week 4)

```
Types session. Read tasks/PLAN.md, the evidence in notes/evidence/ and the existing headers in include/game/. Pick the most-called class or struct that has no header yet. Write or extend its header in include/game/: layout, bit fields, vtable, and names from strings where they are certain; mark guesses with a comment. Prove it on 2 functions that use it; they must pass tools/gate.sh. Then write tasks/redraft-<name>.md listing every function that touches it, for Qwen to re-draft, and add it to tasks/NEXT.md. Show me the diff and wait for my OK before committing.
```

C9 — Hard functions (Thursday from week 5)

```
Hard-functions session. Read notes/needs-claude/ and pick the 3 notes that
block the most functions or share a pattern. Solve them, following
.claude/skills/match-function. For each pattern you crack, add an idiom entry
to the skill so Qwen can apply it, and write tasks/retry-<idiom>.md listing
the stuck functions likely to share it. Run tools/gate.sh, show me the diff
and wait for my OK before committing.
```

Use Opus (`/model opus`) for C9 only when a function is over 1 KB. Everything else runs on Sonnet.

C10 — Plan next week (Friday from week 4)

```
Plan next week. Read this week's notes/weekly/ files, tasks/NEXT.md and the
counts in build/queue.tsv. Close the finished tasks. Write next week's tasks
for Qwen (factory order, evidence to collect for Tuesday's types session,
retries from the idioms added this week) so Qwen has 7 days of work. Keep
each task small enough for a 28K-token context and give it a pass/fail check.
Change CLAUDE.md only if a rule has to change. Show me the diff and wait for
my OK before committing.
```

Qwen prompts (terminal B)

These stay the same all year. What Qwen actually does changes through the task files Claude writes.

Q1 — Start or resume (every morning, and after any restart)

```
Read CLAUDE.md and tasks/NEXT.md. In build/queue.tsv, return every 'claimed'
row older than 24 hours to 'todo'. Then continue the first open task in
tasks/NEXT.md. When its check passes, mark it done and start the next one.
Append what you do to notes/weekly/<today>-qwen.md. Never push.
```

Q2 — Do one named task

```
Read CLAUDE.md, then do tasks/<file>.md exactly as written. Log results in
the format it asks for. When its check passes, mark it done in tasks/NEXT.md
and tell me.
```

Q3 — Function factory (from week 5)

```
Read CLAUDE.md and tasks/NEXT.md, then run the match-function skill on
build/queue.tsv: claim the next 'todo' function, match it or write its
notes/needs-claude/ note, then take the next one. Do any retry or redraft
tasks in tasks/NEXT.md first. Keep going until I stop you. Every 10
functions, append a one-line count to notes/weekly/<today>-qwen.md.
```

Q4 — Collect type evidence

```
Read CLAUDE.md and do the evidence task listed in tasks/NEXT.md: collect
vtables, constructors, per-object field accesses and assert/__FILE__ strings
into notes/evidence/ as plain data files. Don't change src/.
```

Q5 — End of day (every evening)

```
Stop after the current function. Run tools/gate.sh. If it passes, commit the
new matches in one commit (src/ only) and any tool or task changes in a
separate commit, with accurate counts in the messages. Then write today's
report in notes/weekly/<today>-qwen.md: functions and bytes matched today,
functions marked stuck, needs-claude notes written, and the gate result. Show
me the report. Don't push.
```

Q6 — Pause (before C6, or when Claude needs src/ to itself)

```
Finish or abandon the current function and return your claimed rows in
build/queue.tsv to 'todo'. Stop any permuter or batch job that is running
(check build/permute_batch.pid). Then wait, and don't touch src/ until I say
resume.
```

To resume after Q6, paste Q1 (or Q3 once the factory is running).